Applied Intelligence (2024) 54:1672–1690
https://doi.org/10.1007/s10489-023-05253-5

Adversarial perturbation denoising utilizing common characteristics in deep feature space

Jianchang Huang[1] · Yinyao Dai[1] · Fang Lu[1] · Bin Wang[2] · Zhaoquan Gu[3] · Boyang Zhou[4] · Yaguan Qian[1]

Accepted: 23 December 2023 / Published online: 13 January 2024
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract
Recent studies have shown that deep neural networks (DNNs) are vulnerable to adversarial examples (AEs). Denoising based on the input pre-processing is one of the defenses against adversarial attacks. However, it is hard to remove multiple adversarial perturbations, especially in the presence of evolving attacks. To address this challenge, we attempt to extract the commonality of adversarial perturbations. Due to the imperceptibility of adversarial perturbations in the input space, we conduct the extraction in the deep feature space where the perturbations become more apparent. Through the obtained common characteristics, we craft common adversarial examples (CAEs) to train the denoiser. Furthermore, to prevent image distortion while removing as much of the adversarial perturbation as possible, we propose a hybrid loss function that guides the training process at both the pixel level and the deep feature space. Our experiments show that our defense method can eliminate multiple adversarial perturbations, significantly enhancing adversarial robustness compared to previous state-of-the-art methods. Moreover, it can be plug-and-play for various classification models, which demonstrates the generalizability of our defense method.

Keywords Deep neural network · Adversarial examples · Deep feature · Denoiser · Adversarial robustness

1 Introduction

Deep neural networks (DNNs) have been widely used in image classification [1, 2], object detection [3], and natural language processing [4]. However, a sincere vulnerability exists in DNNs, which can be misclassified by *adversarial examples* (AEs)[2]. The so-called adversarial examples are crafted by adding well-designed perturbations to natural examples [2, 5]. Although the perturbation size is tiny and nearly invisible to human eyes, it can easily mislead DNNs to make incorrect predictions. Recently, adversarial examples have become a significant security issue in real-world applications, such as self-driving [6, 7] and identity recognition [8, 9]. Therefore, it becomes essential to protect DNNs against adversarial examples.

Many countermeasures against adversarial examples have been proposed in recent years. There are main two categories of defense methods against adversarial examples: (1) active defense and (2) passive defense. Active defense involves modifying the model parameters to enhance robustness against adversarial examples [5, 10–13]. Passive defense aims to reduce the impact of adversarial perturbations on network classification by modifying the input images [14–18]. Previous works have two directions for designing passive defense: input denoising and feature denoising. The first direction aims to partially or completely eliminate adversarial perturbations from the inputs, while the second direction aims to mitigate the impact of adversarial perturbations on the high-level features learned by DNNs.

In this paper, we focus on passive defenses because they can be applied to various classification models after being trained, resulting in a significant reduction in computational complexity. Guo et al. [17] proposed two methods for transforming the inputs. These input-based transformation defense methods appear to be ineffective in countering evolving adversarial examples. Xu et al. [19] first utilized feature squeezing for denoising adversarial perturbations. However, He et al. [20] demonstrated that this method is still vulnerable to an adaptive knowledgeable adversary. A generative adversarial network (GAN) is a powerful tool for learning a generative model for data distribution. Defense-GAN [21] and Adversarial Perturbation Elimination GAN (APE-GAN)

✉ Yaguan Qian
qianyaguan@zust.edu.cn

Extended author information available on the last page of the article

[22] are two typical algorithms among all these works. However, they were defeated by the recent attacks [23].

Recently, auto-encoder-based input denoising has been proven to be a promising defense. Meng et al. [24] proposed a defensive system called MagNet, which consists of a detector and a reformer. In MagNet, an auto-encoder is used to learn the manifold of natural examples. The detector distinguishes between natural and adversarial examples by analyzing the relationships between these examples and the learned manifold. The reformer is designed to transform adversarial examples into benign examples. However, Carlini et al. [25] demonstrated that MagNet is vulnerable to transferable adversarial examples generated by the C&W attack. Liao et al. [18] regarded the imperceptible perturbations as noise and design a high-level representation-guided denoiser (HGD). However, HGD still requires specific adversarial examples during the training of the denoiser, which poses challenges in defending against novel adversarial attacks.

In conclusion, the previous passive defense methods have two main issues: (1) difficulty in defending against unknown attacks; and (2) impact on the classification accuracy of natural examples. To address these issues, we investigated the commonality of adversarial examples. However, it is a challenging task to perform in the input space due to the imperceptible nature of adversarial perturbations at the pixel level. Intuitively, these perturbations will undergo significant changes in the deep feature space. Figure 1 presents the similarity between the deep features of the adversarial examples and the deep features of the natural examples. We observe that adversarial examples are easily recognized and have a commonality in deep feature space. Specifically, adversarial perturbations not only disrupt the deep features of natural examples but also make their deep features progressively more dissimilar to their original ones. This inspires us that once the denoiser eliminates the commonality of adversarial examples, it can defend against multiple adversarial attacks. First, we exploit this commonality to craft *common adversarial examples* (CAEs) by actively maximizing the distortion of the deep features of natural examples. Then, we choose an encoder-decoder structure for our denoiser and train the denoiser using CAEs as the training set. Instead of using a special set of adversarial examples as a training set for the denoiser, we enable the denoiser to remove the commonality of the adversarial examples. This enables the denoiser to effectively defend against unknown adversarial attacks. Finally, to prevent image distortion while removing as much of the adversarial perturbation as possible, we propose a hybrid loss function that guides the training process at both the pixel level and the deep feature space.

Our main contributions are summarized as follows:

- We observe that adversarial examples are easily recognized and have a commonality in deep feature space. To defend against unknown adversarial attacks, we exploit this commonality to craft CAEs by actively maximizing the distortion of the deep features of natural examples. Additionally, they are used to train the denoiser.
- To prevent image distortion while removing as much of the adversarial perturbation as possible, we propose a hybrid loss function to guide the training of the denoiser at both the pixel level and the deep feature space.
- Extensive experiments are conducted on CIFAR-10, CIFAR-100, SVHN, and ImageNet-30. The results show that compared to previous state-of-the-art approaches, our defense method can eliminate various adversarial perturbations, and maintain the high classification accuracy of the classifier.

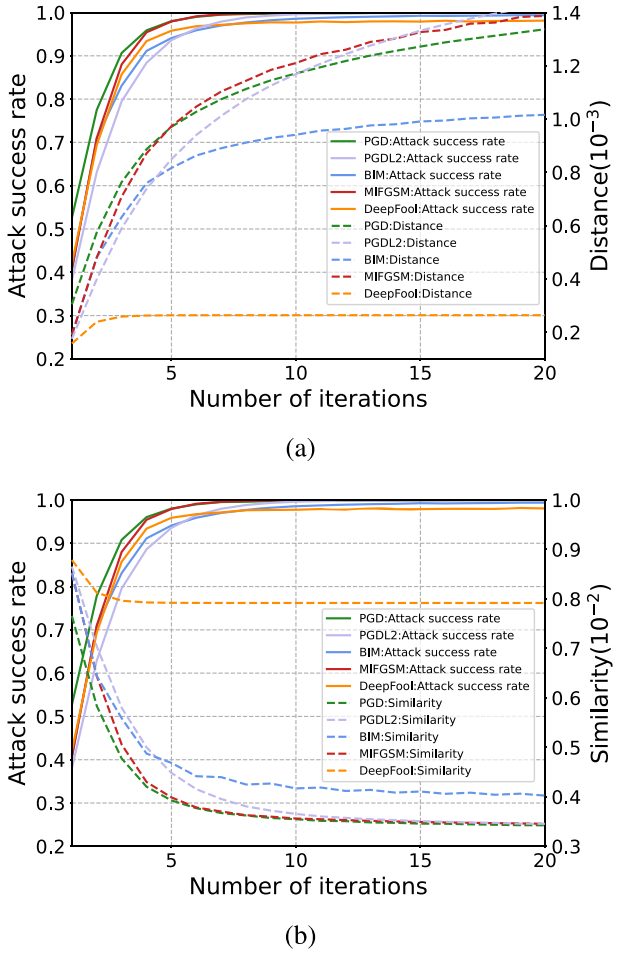

Fig. 1 The attack success rate of adversarial examples and the difference between the deep features of adversarial examples and natural examples against a ResNet-18 [1] target model on *CIFAR-10* [17]. The adversarial examples are respectively crafted by PGD [13], PGDL_2 [13], BIA [10], MI-FGSM [26] and DeepFool [27]. It can be seen from the figure that the difference between the deep features increases as the attack success rate increases. (*left*: measuring similarity of deep features using (3), *right*: measuring similarity of deep features using (4))

2 Related work

2.1 Adversarial attacks

Adversarial examples [2] are maliciously designed inputs with a small difference from natural images but cause the classifier to give wrong classifications. They can be crafted by single-step or multi-step attacks following the direction of adversarial gradients under an L_p norm ($p = 1, 2, \infty$) perturbation budget. They can be crafted by single-step or multi-step attacks following the direction of adversarial gradients under an L_p norm perturbation budget. Attacks based on this strategy include the fast gradient sign method (FGSM) [5], the basic iterative Method (BIM) [10], the strongest first-order information-based projected gradient descent (PGD) method [13], the decoupling direction and norm (DDN) method [28] and the auto-attack (AA) method [29]. To improve transferability, a variant of BIM called momentum iterative fast gradient sign method (MI-FGSM) [26] is proposed. DeepFool [27] finds the smallest perturbation crossing the model decision boundary. C&W [30] solves an optimization problem that minimizes both the objective function and the difference between adversarial and clean images. Rather than optimizing the objective function at a single point, the translation-invariance input diversity method (TI-DIM) [31, 32] uses a set of translated images to optimize an adversarial example. In addition, unlike these pixel-constrained attacks, which do not consider the semantic or geometric information, spatially constrained attacks focus on mimicking non-suspicious vandalism via geometry and spatial transformation, *e.g.*, faster Wasserstein attack (FWA)[33] and spatial transform attack (STA) [34].

2.2 Defense against Adversarial Attacks

Adversarial training, proposed by [5, 13, 35, 36], is an approach to improve the robustness of the target model. Usually, it augments the adversarial examples to the training set in the retraining phase. Adversarial training could achieve good results in defending against white-box and black-box attacks. However, it requires a sufficient amount and variety of adversarial examples, leading to high time complexity. Adversarial training could improve the accuracy of the target model on adversarial examples, but it typically cannot be transferred directly to other models or tasks.

Preprocessing-based methods process the inputs to achieve robustness against adversarial perturbation. Many recent defenses resort to randomization schemes to mitigate adversarial perturbations' effects in the input domain. Such as Xie et al. [37] utilized two random transformations random resizing and padding to mitigate the adversarial effects at the inference time. JPEG compression [17] and total variation minimization (TVM) [17] are proposed to eliminate high-frequency components and small localized changes respectively. Comdefend [38] is an end-to-end image compression model, which utilizes image compression to remove adversarial perturbations or destroy the structure of adversarial perturbations. Many works intend to exploit generative adversarial networks to learn benign data distribution to generate a benign projection for an adversarial input. Defense-GAN [21] trains a generator to model the distribution of benign images. In the testing stage, Defense-GAN cleanses an adversarial input by searching for an image close to the adversarial input. APE-GAN [22] directly learns a generator to cleanse an adversarial example by using it as input, and outputs a benign counterpart. Denoising is a very straightforward method in terms of mitigating adversarial perturbations. Meng et al. introduce a defensive system called MagNet [24]. In MagNet, an auto-encoder is used to learn the manifold of benign examples. A high-level representation guided denoiser (HGD) [18] method is utilized as a pre-processing step to eliminate adversarial noise. Xie et al. [39] designed a block for learning several denoising operations to rectify the features learned by intermediate layers in DNNs. Zhou et al. [40] utilized class activation features to eliminate adversarial perturbations.

Different from the above methods, we explore the commonality of the adversarial examples and use this commonality to train the denoiser to achieve defense against unknown attacks.

3 Methodology

We will first describe our method at a high level here. Our goal is to design a denoising defense method to remove multi-types of adversarial perturbations and provide robust protections. To achieve this goal, we need to explore the commonality of adversarial examples and exploit this commonality to achieve defense against unknown adversarial attacks. However, it is difficult to find the commonality of different types of adversarial examples in the input space, since they are too tiny to detect. Through experiments, we observe that adversarial examples are easily recognized and have a commonality in deep feature space (described in Section 3.1).

This inspired us to achieve defense against unknown adversarial attacks once our denoiser can eliminate this commonality of adversarial examples. For this reason, we actively exploit the commonality of adversarial examples to craft CAEs (described in Section 3.2). We then use CAEs to train the denoiser, enabling it to eliminate this commonality and achieve defense against unknown adversarial attacks (described in Section 3.3).

Figure 2 illustrates the whole process to train the denoiser. Let F be a feature extractor, P be a classification model,

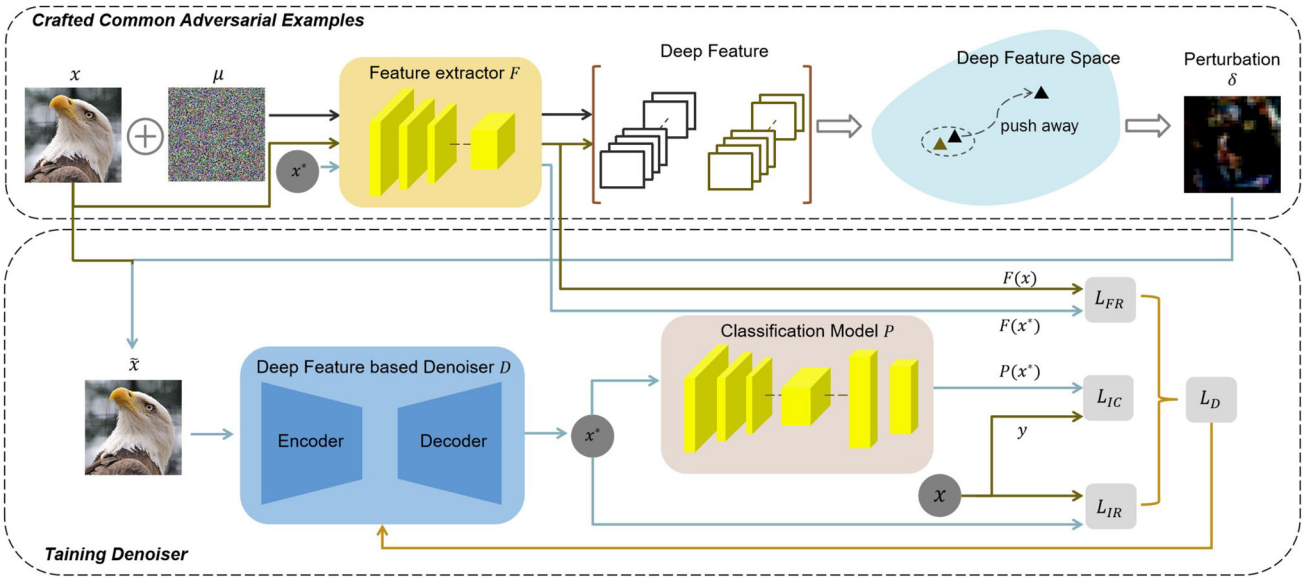

Fig. 2 A visual illustration of our defense method. x denotes natural example, μ denotes random perturbations, $\widetilde{x}$ denotes a common adversarial example, x^* denotes the denoised example, F is a feature extractor, $F(x)$ and $F(x^*)$ denote the deep feature of x and x^* respectively, D is a denoiser, P is a classification model. We first craft the perturbation δ by keeping $F(x_\mu)$ maximally away from $F(x)$ and then add δ to x, namely *common adversarial example*. Finally, we use them to train the denoiser

and D represent a denoiser. For a natural example x, we add random perturbations to it and obtain a perturbed example x_μ. The feature extractor F obtains their deep feature $F(x)$ and $F(x_\mu)$, respectively. Furthermore, we craft the perturbation δ by keeping $F(x_\mu)$ as far away from $F(x)$ as possible. Then, we crafted CAEs $\widetilde{x}$ by adding perturbation δ to x. Besides, we forward $\widetilde{x}$ to D to obtain x^* and forward x^* respectively to F and P to obtain $F(x^*)$ and $P(x^*)$. Finally, we train the denoiser using a hybrid loss function, and this training is independent of the special adversarial examples, such as FGSM.

3.1 Motivation

In this paper, we design a denoiser D, which is structured as an encoder-decoder, to remove adversarial perturbations. The encoder is to extract robust features from adversarial examples, and the decoder is to reconstruct clean examples based on these robust features. Let x denote the natural example and y is its ground-truth label. The denoiser is expressed by a function $D : x' \rightarrow x^*$, where x' and x^* represent the adversarial and denoised examples, respectively. The denoiser aims to minimize the difference between x^* and x. Thus, the loss function for training the denoiser is defined as follows:

$$\mathcal{L} = Dist(x^*, x) \tag{1}$$

where $Dist(\cdot, \cdot)$ represents the distance between x^* and x. The ideal result is $P(x^*) = P(x) = y$ after the denoiser is trained, where P is the target classifier. Previous work generally used a specific attack (*e.g.*, PGD) to generate adversarial examples to train the denoiser. However, this situation leads to the denoiser being trained on special adversarial examples. In other words, it is difficult for a denoiser like this to eliminate unknown adversarial perturbations.

To address this issue, we aim to build a defense method to defend against more unknown attacks. For this reason, we explored the commonality of adversarial examples in DNNs. However, it is nontrivial to conduct in the input space since the adversarial perturbations are imperceptible at the pixel level. In other words, it is hard to characterize the adversarial perturbations in the input space. Intuitively, we can reasonably speculate that the adversarial perturbations will change significantly in the deep feature space. Therefore, we attempt to explore the commonality of adversarial examples in the deep feature space.

To confirm our speculations, we conduct a proof-of-concpet experiment as shown in Fig. 1. Specifically, we implement a variety of adversarial attacks (*i.e.*, PGD[13], PGD(L_2)[13], BIM[10], MI-FGSM[26], and DeepFool[27]), and then we forward natural examples x and adversarial examples x' to the feature extractor F to obtain their deep features $F(x) \in \mathbb{R}^{m \times n \times c}$ and $F(x') \in \mathbb{R}^{m \times n \times c}$, where m and n are the height and width of the deep feature, c is the number of channels.

To compare the differences in deep features, we define feature similarity to measure the difference between x and x' as follows:

$$\Delta(x', x) = \mathcal{S}(F(x'), F(x)) \tag{2}$$

We use the Frobenius paradigm to measure the similarity of deep features in Fig. 1a :

$$\begin{aligned}\mathcal{S}(F(x'), F(x)) &= \|F(x'), F(x)\|_F \\ &= \sqrt{\sum_{k=1}^{c}\sum_{i=1}^{m}\sum_{j=1}^{n}\left|f_{ij}^{k}(x') - f_{ij}^{k}(x)\right|^2}\end{aligned} \tag{3}$$

where $f_{ij}^{k}(x)$ are the elements of $F(x)$.

Since different distance measures focus on different aspects of feature differences, and measure feature differences on multiple scales (in Fig. 1b), cosine similarity is used to measure the similarity:

$$\begin{aligned}\mathcal{S}(F(x'), F(x)) &= F(x') \cdot F(x) / \left(\left\|F(x')\right\| \|F(x)\|\right) \\ &= \sum_{k=1}^{c}\sum_{i=1}^{m}\sum_{j=1}^{n} f_{ij}^{k}(x') \cdot f_{ij}^{k}(x) / \\ &\left(\sqrt{\sum_{k=1}^{c}\sum_{i=1}^{m}\sum_{j=1}^{n} f_{ij}^{k}(x')^2} \cdot \sqrt{\sum_{k=1}^{c}\sum_{i=1}^{m}\sum_{j=1}^{n} f_{ij}^{k}(x)^2}\right)\end{aligned} \tag{4}$$

As shown in Fig. 1, we can see that the similarity between the deep features of the adversarial examples and the deep features of the natural examples decreases as the success rate of the attack on the adversarial examples increases. Furthermore, adversarial examples have a common characteristic in DNNs. Adversarial perturbations not only disrupt the deep features of natural examples but also make their deep features progressively more dissimilar to their original selves. These observations confirm our speculation. We can leverage this commonality of adversarial examples to achieve defense against unknown adversarial attacks. Once our denoiser can eliminate the commonality of the adversarial examples, it will be able to remove unknown adversarial perturbations. For this purpose, we can design special adversarial examples by actively maximizing (2) and use them to establish an effective defense strategy. By doing so, it can repair the distortion of deep features caused by various adversarial perturbations.

3.2 Crafting common adversarial examples

To defend against various unknown attacks, we utilize the commonality of adversarial examples (*i.e.*, CAEs) to craft training examples in deep feature space to train the denoiser.

Algorithm 1 Craft Common adversarial Examples.

Input: A pre-trained model F, natural example x, perturbation budget ϵ, number of iterations T and attack step size α, random perturbation μ;
Output: An deep feature training example $\widetilde{x}$

```
x̃_0 ← x + μ
for t = 0 to T − 1 do
    Forward x and x̃_t to F, and obtain deep feature F(x) and F(x̃_t)
    Computer the feature similarity Δ(x̃_t, x) using (2)
    Computer gradients : g_t = ∇_x 1(x̃_t, x)
    Computer perturbation : δ_t = α sign (g_t)
    Update the adversarial example x̃_t : x̃_{t+1} = x̃_t + δ_t
    Project x̃_{t+1} to the vicinity of x : x̃_{t+1} = Clip(x̃_{t+1}, x−ϵ, x+ϵ)
end for
return x̃
```

Since adversarial perturbations distort deep features. On the contrary, we craft CAEs by maximally distorting the deep features of natural examples and train our denoiser with them so that our denoiser can eliminate the effect of perturbations on deep features. We achieve defense against unknown adversarial attacks by correcting the distortion of deep features by perturbations. In the following, we describe how we distort the deep features of natural examples.

Given a pre-trained feature extractor F, we forward natural examples x to F and obtain $F(x)$, $F(x)$ denotes the deep features of x extracted by F. The degree of distortion between deep features is measured according to (2). Based on the above description, we design a deep feature-based attack method to craft CAEs. Our method aims to solve the following optimization problem to craft CAEs to train our denoiser through deep feature space:

$$\max_{\widetilde{x}} \Delta(\widetilde{x}, x) \qquad \text{s.t. } \|\widetilde{x} - x\|_\infty \le \epsilon \tag{5}$$

where ϵ denotes the budget of perturbation and $\widetilde{x}$ denotes training examples. We use two measures to calculate the similarity of deep features, $\Delta(\widetilde{x}, x) = \|F(x'), F(x)\|_F$ and $\Delta(\widetilde{x}, x) = (F(\widetilde{x}) \cdot F(x)) / (\|F(\widetilde{x})\| \cdot \|F(x)\|)$. Our method is summarized in Algorithm 1. Given natural examples x, we first initialize $\widetilde{x}_0$ as x plus random perturbation μ. Then, we forward x and $\widetilde{x}_t$ to the pre-trained feature extractor F and obtain their deep features $F(x)$ and $F(\widetilde{x}_t)$. Next, we compute the differences of deep features by $\Delta(\widetilde{x}, x)$ and obtain its gradients. Finally, we take the gradients to update $\widetilde{x}_t$ and obtain $\widetilde{x}_{t+1}$. By iteratively executing such an update procedure, Algorithm 1 could maximize $\Delta(\widetilde{x}, x)$ and output a common adversarial example $\widetilde{x}$.

3.3 Deep feature denoising

An ideal denoiser needs to satisfy the following two criteria. First, the denoiser can effectively eliminate various adversarial perturbations. Second, the denoiser should have minimal

impact on the classification accuracy of natural examples. We have designed a deep feature-based denoiser D to eliminate adversarial perturbations. To address the first point, we utilize the CAEs suggested in section 3.2 as the training data for the denoiser. To address the second point, we need to consider this problem beyond just the pixel level. Therefore, we propose a hybrid loss function that includes a pixel-level image reconstruction loss function, a feature-level feature reconstruction loss function, and a classification-level image classification loss function.

Image Classification Loss. The denoiser should ensure that the reconstruction of the image does not alter the distribution of natural examples in the model. If the denoiser D successfully removes noise from an example and the denoised example x^* can be classified correctly, it should have the same real label y as its corresponding natural example. To achieve this goal, we introduce the following cross-entropy loss function when training D :

$$\mathcal{L}_{\mathcal{IC}} = -\sum_{i=1}^{C-1} y_i \log p_i \tag{6}$$

where $p = \left[p_0, \cdots, p_{c-1}\right]$ is a probability distribution, and each element p_i denotes the probability that the example belongs to the i-th class; $y = \left[y_0, \cdots, y_{c-1}\right]$ represents the one-hot representation of the label. In this representation, $y_i = 1$ when the example belongs to category i, and $y = 0$ otherwise. C is the number of classes.

Image Reconstruction Loss. The denoised example should closely resemble its corresponding natural example in pixel space. We choose the L_2 distance to calculate the similarity between examples. Our objective for the image reconstruction loss function is to encourage the denoising model D to restore the crafted CAEs into natural examples. This is achieved by minimizing the following function:

$$\begin{aligned}\mathcal{L}_{\mathcal{IR}} &= \|x^* - x\|_F \\ &= \sqrt{\sum_{c=1}^{3}\sum_{i=1}^{h}\sum_{j=1}^{w}\left|x^{*k}_{ij} - x^k_{ij}\right|^2}\end{aligned} \tag{7}$$

where $x \in \mathbb{R}^{3\times h\times w}$.

Feature Reconstruction Loss. In addition to the previous pixel-space objective, it is important to ensure that the deep features of denoised examples x^* are similar to the deep features of the natural examples x. This is because the goal is to maximize the distortion of the deep features of natural examples to craft CAEs. Subsequently, our feature reconstruction objective is to encourage D to minimize the following loss function:

$$\mathcal{L}_{\mathcal{FR}} = \delta(F(x^*) - F(x)) \tag{8}$$

where $\delta(F(x^*)-F(x)) = 1-F(x^*)\cdot F(x)/\left(\|F(x^*)\|\,\|F(x)\|\right)$ when we use cosine similarity to measure the similarity of deep features and $\delta(F(x^*) - F(x)) = \|F(x^*) - F(x)\|_F$ when we use Frobenius paradigm to do so.

Combining the above image reconstruction loss, feature reconstruction loss, and image classification loss, the overall loss function for D is given as:

$$\mathcal{L}_{\mathcal{D}} = \mathcal{L}_{\mathcal{IC}} + \lambda_1\mathcal{L}_{\mathcal{IR}} + \lambda_2\mathcal{L}_{\mathcal{FR}} \tag{9}$$

where λ_1 and λ_2 are positive parameters to trade off each component. Given natural examples x, we first craft CAEs $\widetilde{x}$. Then, we forward-pass $\widetilde{x}$ through the denoising model D to obtain x^* and calculate $\mathcal{L}_{\mathcal{IR}}$. Next, we forward-pass x^* through F to obtain $F(x^*)$ and P to obtain $P(x^*)$ and respectively calculate $\mathcal{L}_{\mathcal{FR}}$ and $\mathcal{L}_{\mathcal{IC}}$. Finally, we take a gradient step to update D adopted to minimize $\mathcal{L}_{\mathcal{D}}$. The above steps are repeated until D converges.

4 Experiments

First, we introduce the datasets, network architectures, and training details used in this paper (Section 4.1). Then, we present and analyze the experimental results of defending against unseen types of attacks. Moreover, we demonstrate the applicability of our defense approach (Section 4.2). Finally, we evaluate the robustness of our defense method to perturbation budgets. We also explore whether the same regularity is found in specific image classes and different network architectures (Section 4.3).

4.1 Experiments setup

Datasets. We verify the effectiveness of our defense method on four popular benchmark datasets, *i.e.*, CIFAR-10 [17], CIFAR-100 [17], SVHN [42], and ImageNet-30 [43]. CIFAR-10 and SVHN both have ten classes of images, but the former contains 60,000 training images and 10,000 test images, and the latter contains 73,257 training images and 26,032 test images; CIFAR-100 has 100 classes, containing the same number of training (testing) examples as CIFAR-10; ImageNet-30 is a subset with 30 classes extracted randomly from the ImageNet dataset. Images in the four datasets are all regarded as natural examples. Adversarial examples for evaluating defense models are crafted by applying state-of-the-art attacks. These attacks include: (i) pixel-constrained attacks, *i.e.*, FGSM [5], PGD [13], PGD(L_2) [13], C&W [30], AutoAttack(AA) [29], DeepFool [27], BIM [10], DDN [28], MI-FGSM [26], SI-NI-FGSM [44], and VMI-FGSM [45], and (ii) spatially-constrained attacks, *i.e.*, STA [34]. Pixel-constrained attacks manipulate pixel values directly by leveraging the L_p norm distance to penalize adversarial

Table 1 Classification error rates (percentage) of adversarial examples crafted by various pixel-constrained and spatially-constrained attacks against the ResNet-18 target model on *CIFAR-10* (lower is better)

Defense	Attacks											
	None	FGSM	PGD	PGD(L_2)	DeepFool	BIM	AA	MI-FGSM	DDN	STA	SI-NI-FGSM	VMI-FGSM
Trades [41]	18.43	33.48	16.72	16.87	16.27	16.79	16.76	16.75	16.07	19.86	23.63	38.01
JPEG(50) [17]	21.54	41.77	29.83	32.29	22.49	26.16	27.83	29.14	20.42	40.68	36.83	44.54
JPEG(75) [17]	15.23	37.41	43.87	41.58	18.11	25.76	42.65	37.04	14.21	43.34	61.66	73.15
TVM [17]	9.65	34.61	61.79	63.37	40.52	42.28	66.46	42.56	18.15	27.97	35.78	40.80
Defense-GAN [21]	**5.92**	36.23	29.01	35.72	12.49	32.75	40.79	32.99	9.56	23.51	33.02	60.59
APE-GAN [22]	11.81	39.43	41.47	46.57	16.10	30.34	33.86	39.07	11.07	26.69	43.27	73.65
HGD [11]	6.06	**11.64**	15.91	10.61	**2.89**	17.89	12.86	14.92	**2.01**	18.39	32.53	51.54
ComDefend [38]	12.46	39.23	46.84	55.92	14.66	35.76	36.13	47.47	7.99	18.64	34.61	58.72
DFD	7.61	32.37	**5.25**	**7.84**	3.90	**3.81**	**5.01**	**5.52**	4.00	**15.24**	**19.89**	**34.71**
DFD′	7.58	34.18	6.01	9.08	4.00	4.02	5.45	6.42	3.88	15.48	21.15	37.53

The compression quality of JPEG is 50 and 75, and the weight of TVM is 0.003. Both DFD and DFD′ are our defense methods, which measure the similarity of deep features using (3) and (4), respectively. For each attack, we show the most successful defense with **bold** and the second with underline

Table 2 Classification error rates (percentage) of adversarial examples crafted by various pixel-constrained and spatially-constrained attacks against the ResNet-18 target model on *SVHN* (lower is better)

Defense	Attacks											
	None	FGSM	PGD	PGD(L_2)	DeepFool	BIM	AA	MI-FGSM	DDN	STA	SI-NI-FGSM	VMI-FGSM
Trades [41]	11.07	31.06	12.19	**13.65**	12.13	12.17	12.89	10.41	16.35	11.07	21.56	15.60
JPEG(50) [17]	6.84	89.85	96.69	44.77	96.87	97.67	97.05	6.56	32.77	6.84	86.64	95.74
JPEG(75) [17]	6.62	94.02	99.57	58.33	99.61	99.73	99.58	10.23	38.49	6.62	93.50	98.44
TVM [17]	10.01	90.05	94.53	40.97	93.52	95.48	97.43	21.37	27.76	10.01	39.02	42.25
Defense-GAN [21]	3.51	78.22	63.59	37.25	62.53	60.07	31.06	25.21	22.75	16.73	42.53	47.81
APE-GAN [22]	**3.46**	80.79	64.98	14.57	64.24	63.40	64.00	3.05	23.65	**3.46**	71.53	81.44
HGD [11]	3.63	87.90	62.06	14.54	90.97	46.59	78.25	**2.25**	37.43	3.63	87.13	95.22
Comdefend [38]	3.98	52.51	21.96	23.74	12.53	21.59	21.69	21.04	**13.27**	4.51	33.26	67.82
DFD	5.21	17.89	5.34	16.53	3.54	3.69	3.34	3.78	17.66	4.77	11.11	12.76
DFD′	5.49	**8.14**	**2.94**	18.12	**2.47**	**2.39**	**2.52**	3.59	15.97	5.49	**6.94**	**5.49**

The compression quality of JPEG is 50 and 75, and the weight of TVM is 0.003. Both DFD and DFD′ are our defense methods, which measure the similarity of deep features using Eqs. (3) and (4), respectively. For each attack, we show the most successful defense with **bold** and the second with underline

Table 3 Classification error rates (percentage) of adversarial examples crafted by various pixel-constrained and spatially-constrained attacks against the ResNet-18 target model on *CIFAR-100* (lower is better)

Defense	Attacks											
	None	FGSM	PGD	PGD(L_2)	DeepFool	BIM	AA	MI-FGSM	DDN	STA	SI-NI-FGSM	VMI-FGSM
Trades [41]	44.25	**40.10**	35.37	35.31	35.14	35.49	35.02	35.59	33.69	38.15	46.66	46.52
JPEG(50) [17]	43.14	44.62	36.14	36.21	34.19	36.41	35.43	36.82	32.66	40.10	50.78	51.20
JPEG(75) [17]	39.72	54.43	38.41	38.09	30.81	38.95	36.36	39.95	29.08	38.37	48.86	50.79
TVM [17]	29.06	49.74	54.57	53.97	47.57	55.12	51.99	55.50	36.89	42.19	30.78	35.89
Defense-GAN [21]	**21.73**	76.56	65.23	54.21	30.02	54.89	56.09	62.21	62.31	44.91	57.62	62.77
APE-GAN [22]	38.90	81.39	73.78	69.94	32.74	72.91	65.48	75.60	29.23	45.72	84.00	84.54
HGD [11]	21.77	69.03	41.66	43.96	54.14	43.03	39.43	36.25	**6.35**	40.60	95.74	99.84
Comdefend [38]	39.07	63.33	45.49	46.25	27.57	48.37	38.32	50.57	25.53	33.65	64.02	69.56
DFD	26.98	65.72	**17.02**	**23.56**	10.98	**19.64**	**14.06**	**25.02**	11.25	28.56	**40.92**	**42.45**
DFD′	27.69	66.99	21.72	26.47	**10.89**	24.94	16.26	31.08	9.94	**27.03**	44.66	46.24

The compression quality of JPEG is 50 and 75, and the weight of TVM is 0.003. Both DFD and DFD′ are our defense methods, which measure the similarity of deep features using Eqs. (3) and (4), respectively. For each attack, we show the most successful defense with **bold** and the second with underline

Table 4 Classification error rates (percentage) of adversarial examples crafted by various pixel-constrained and spatially-constrained attacks against the ResNet-18 target model on *ImageNet-30* (lower is better)

Defense	Attacks											
	None	FGSM	PGD	PGD(L_2)	DeepFool	BIM	AA	MI-FGSM	DDN	STA	SI-NI-FGSM	VMI-FGSM
Trades [41]	26.72	25.14	23.26	24.67	22.21	23.41	22.90	23.04	22.17	26.28	23.79	<u>23.34</u>
JPEG(50) [17]	28.93	42.97	35.70	28.42	24.78	36.21	34.67	36.14	24.36	28.63	46.32	42.36
JPEG(75) [17]	28.87	54.56	41.62	28.08	25.22	41.99	41.33	42.28	25.09	33.33	48.97	48.06
TVM [17]	31.61	34.51	28.82	29.58	27.61	29.63	29.26	29.48	26.99	29.52	33.87	31.16
Defense-GAN [21]	10.93	53.74	42.75	40.51	36.56	60.24	47.59	57.46	25.89	35.91	60.51	72.59
APE-GAN [22]	**8.97**	87.43	89.51	28.33	11.07	89.36	96.56	88.17	5.34	18.80	92.64	99.05
HGD [11]	<u>9.13</u>	75.97	89.79	14.67	10.33	89.41	90.43	86.11	**1.59**	17.69	91.90	99.86
Comdefend [38]	17.73	43.06	53.04	9.92	6.74	52.16	40.23	55.30	6.29	**10.13**	62.54	76.57
DFD	9.87	<u>18.89</u>	<u>14.67</u>	<u>6.83</u>	<u>5.02</u>	<u>12.23</u>	<u>13.01</u>	<u>16.27</u>	<u>2.37</u>	<u>13.29</u>	<u>15.17</u>	28.24
DFD′	11.67	**13.72**	**10.31**	**6.50**	**4.84**	**9.44**	**9.58**	**12.73**	4.32	13.58	**12.30**	**21.43**

The compression quality of JPEG is 50 and 75, and the weight of TVM is 0.003. Both DFD and DFD′ are our defense methods, which measure the similarity of deep features using Eqs. (3) and (4), respectively. For each attack, we show the most successful defense with **bold** and the second with <u>underline</u>

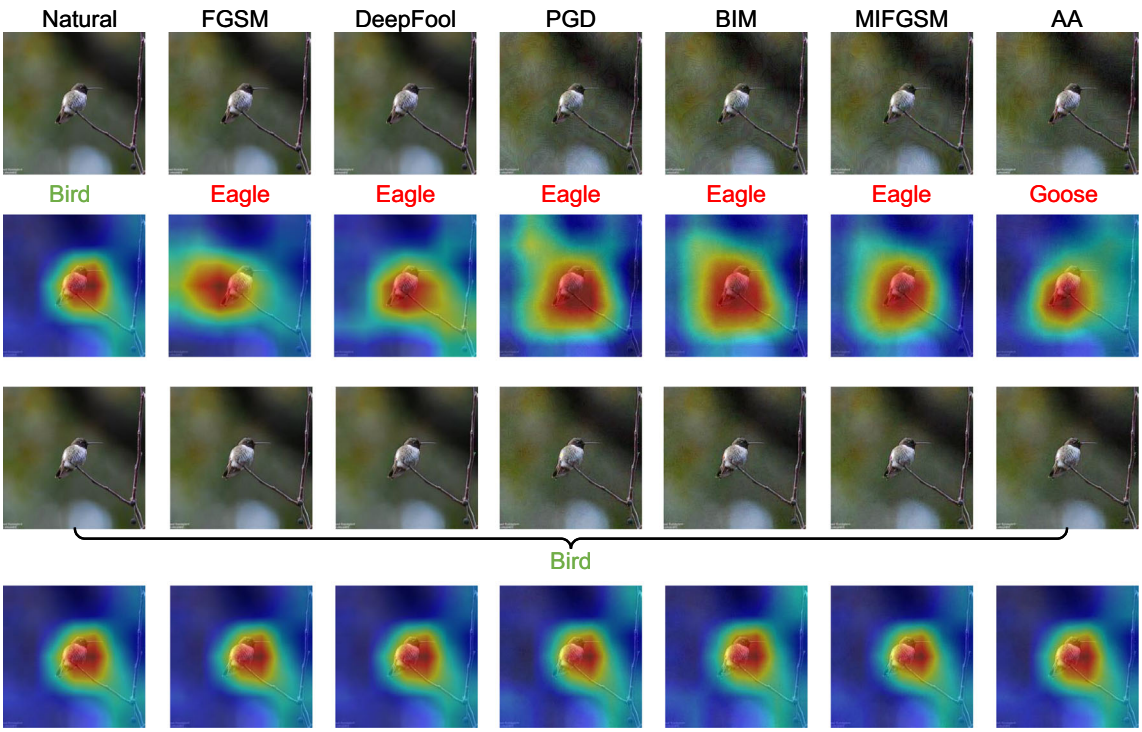

Fig. 3 A visual illustration of the defense effect of our defense method. Our defense method can eliminate multiple adversarial perturbations and preserve the critical information of the image. (first row: adversarial examples; second row: class activation maps of adversarial; third row: restored examples; last row: class activation maps of restored examples)

perturbation. In contrast, spatially-constrained attacks focus on mimicking non-suspicious vandalism via spatial transformation and physical modifications [46]. In experiments, we selected correctly classified images to evaluate various attacks, ensuring that the effectiveness of attacks is caused by the attacks themselves and not the model performance.

Table 5 Classification error rates (percentage) of attacks against distinct target models with DFD and DFD′ (lower is better) on *CIFAR-10*

Target Model	Defense	Attacks						
		PGD	C&W	DeepFool	AA	MI-FGSM	SI-NI-FGSM	VMI-FGSM
ResNet50	None	100	100	95.96	100	100	98.09	100
	APE-GAN [22]	58.25	18.26	14.82	50.07	59.78	46.03	77.40
	HGD [11]	22.18	**7.80**	7.56	19.69	23.11	27.49	52.73
	DFD	**16.72**	7.82	**7.37**	**14.29**	**17.01**	**24.17**	**47.60**
	DFD′	17.23	8.53	7.40	14.97	18.61	25.98	52.78
WideResNet	None	100	99.99	98.2	100	100	98.01	99.51
	APE-GAN [22]	35.82	19.69	18.93	36.69	37.24	40.71	67.55
	HGD [11]	20.75	7.83	7.59	21.41	20.98	31.91	48.13
	DFD	**9.26**	**6.88**	**6.98**	9.42	**10.23**	**22.72**	**39.79**
	DFD′	9.93	7.94	7.08	**8.82**	10.33	24.01	42.60
VGG16	None	100	99.97	99.44	100	100	94.64	98.97
	APE-GAN [22]	46.53	21.45	18.93	46.93	48.81	45.83	74.19
	HGD [11]	28.37	**13.84**	17.41	28.39	30.13	37.86	61.79
	DFD	**15.04**	15.07	**16.26**	**9.01**	**16.38**	**28.06**	**49.75**
	DFD′	16.04	14.63	17.88	9.80	17.91	29.03	52.51

We transfer DFD and DFD′, which uses the ResNet-18 network as the pretraining network, to the ResNet-50, Wide-ResNet, and VGG-16 classification models. For each attack, we show the most successful defense with **bold** and the second one with underline

Table 6 Classification error rates (percentage) of attacks against distinct target models with DFD and DFD′ (lower is better) on *SVHN*

Target Model	Defense	Attacks						
		PGD	C&W	DeepFool	AA	MI-FGSM	SI-NI-FGSM	VMI-FGSM
ResNet50	None	99.72	99.98	98.29	99.93	99.76	87.59	98.40
	APE-GAN [22]	79.28	90.38	82.73	93.59	80.17	67.87	83.63
	HGD [11]	92.14	75.62	14.64	90.91	92.46	78.31	93.28
	DFD	10.07	9.54	**13.09**	8.71	10.87	17.38	22.81
	DFD′	**8.05**	**9.39**	15.27	**7.39**	**8.40**	**12.79**	**15.28**
WideResNet	None	99.54	99.97	98.65	99.73	99.55	88.38	97.90
	APE-GAN [22]	84.65	10.81	17.64	81.74	85.32	87.09	74.82
	HGD [11]	95.84	16.02	15.67	94.04	96.03	82.84	94.78
	DFD	10.18	9.26	**14.37**	**9.21**	10.32	18.93	20.55
	DFD′	**7.31**	**8.63**	14.45	10.61	**7.14**	**12.70**	**11.88**
VGG16	None	99.58	99.75	98.97	99.76	99.61	85.20	98.48
	APE-GAN [22]	89.31	91.25	75.66	95.23	99.6	99.69	99.83
	HGD [11]	97.75	12.38	**26.88**	96.75	97.76	82.36	96.69
	DFD	16.62	10.54	31.09	14.39	**7.56**	23.53	31.18
	DFD′	**12.39**	**10.17**	33.31	**10.89**	9.61	**18.35**	**22.70**

We transfer DFD and DFD′, which uses the ResNet-18 network as the pretraining network, to the ResNet-50, Wide-ResNet, and VGG-16 classification models. For each attack, we show the most successful defense with **bold** and the second one with underline

Network architectures. We use three network architectures to perform classification tasks on CIFAR-10, CIFAR-100, SVHN, and ImageNet-30, *i.e.*, ResNet [1], VGG [47], and Wide-ResNet [34]. The depth and widen factor in Wide-ResNet are set to 34 and 10 on CIFAR-10, CIFAR-100, and SVHN. The depth and width factor in Wide-ResNet are set to 101 and 2 on ImageNet-30. The architecture of our denoiser is a DUNET architecture [18]. It consists of multiple basic blocks and each block contains a 3×3 convolutional layer, a batch normalization layer, and a rectified linear unit.

Training details. For a fair comparison, all experiments on CIFAR-10, CIFAR-100, and SVHN are conducted on NVIDIA RTX 2080 GPU; all experiments on ImageNet-30 are conducted on NVIDIA RTX 3090 GPU and all methods

Table 7 Classification error rates (percentage) of attacks against distinct target models with DFD and DFD′ (lower is better) on *CIFAR-100*

Target Model	Defense	Attacks						
		PGD	C&W	DeepFool	AA	MI-FGSM	SI-NI-FGSM	VMI-FGSM
ResNet50	None	100	100	93.89	100	100	98.46	99.97
	APE-GAN [22]	85.86	42.73	33.54	77.42	86.46	82.50	95.14
	HGD [11]	97.23	36.71	**21.23**	94.67	97.13	91.89	99.49
	DFD	**38.29**	26.85	26.31	**33.59**	41.00	**48.93**	**67.55**
	DFD′	38.48	**26.61**	25.00	33.79	**39.33**	51.19	70.17
WideResNet	None	100	99.98	95.01	100	100	98.76	99.75
	APE-GAN [22]	73.86	43.73	34.76	68.73	73.01	77.34	91.94
	HGD [11]	93.23	40.18	**20.13**	87.91	94.41	93.33	99.17
	DFD	**33.21**	**27.99**	26.87	**29.69**	37.01	**48.46**	**67.17**
	DFD′	33.35	28.03	25.16	32.46	**36.90**	48.99	67.82
VGG16	None	100	100	97.49	100	99.99	97.51	99.71
	APE-GAN [22]	85.77	49.03	41.84	81.61	86.18	84.78	95.74
	HGD [11]	98.61	53.70	32.63	97.69	98.75	94.47	99.25
	DFD	**41.10**	34.21	28.16	35.07	43.21	**58.36**	**79.54**
	DFD′	42.82	**32.97**	**27.78**	**33.18**	**42.17**	59.76	81.12

We transfer DFD and DFD′, which uses the ResNet-18 network as the pretraining network, to the ResNet-50, Wide-ResNet, and VGG-16 classification models. For each attack, we show the most successful defense with **bold** and the second one with underline

Table 8 Classification error rates (percentage) of attacks against distinct target models with DFD and DFD′ (lower is better) on *ImageNet-30*

Target Model	Defense	Attacks						
		PGD	C&W	DeepFool	AA	MI-FGSM	SI-NI-FGSM	VMI-FGSM
ResNet50	None	100	99.87	98.53	100	100	99.60	100
	APE-GAN [22]	99.33	15.47	13.60	94.20	97.27	93.53	99.27
	HGD [11]	95.29	**10.56**	10.59	93.42	95.55	90.47	99.67
	DFD	40.35	11.67	**10.13**	29.68	41.58	31.73	54.53
	DFD′	**33.53**	11.40	13.33	**26.13**	**36.07**	**27.00**	**42.73**
WideResNet	None	100	99.93	95.67	100	100	99.67	100
	APE-GAN [22]	97.60	15.73	13.93	95.60	98.0	93.53	99.33
	HGD [11]	98.53	**9.67**	10.20	96.33	98.40	91.93	99.80
	DFD	40.02	11.20	**10.00**	33.80	40.57	34.87	60.73
	DFD′	**36.67**	11.07	12.00	**28.27**	**39.40**	**33.13**	**54.60**
VGG16	None	92.54	99.28	93.33	99.93	99.33	97.00	83.47
	APE-GAN [22]	87.67	13.60	12.07	83.27	87.27	83.00	81.47
	HGD [11]	82.57	**10.35**	**10.42**	75.86	82.83	80.67	80.73
	DFD	42.25	11.78	11.89	34.63	42.99	36.20	56.07
	DFD′	**38.67**	11.73	11.47	**31.73**	**40.60**	**36.00**	**56.00**

We transfer DFD and DFD′, which uses the ResNet-18 network as the pretraining network, to the ResNet-50, Wide-ResNet, and VGG-16 classification models. For each attack, we show the most successful defense with **bold** and the second one with underline

are implemented by PyTorch. We use the implementation codes of FGSM PGD, PGD(L_2), C&W, DeepFool, BIM, AA, MI-FGSM, SI-NI-FGSM, and VMI-FGSM in the torchattacks toolbox [48] and the implementation codes of DDN and STA methods in the advertorch toolbox [49]. The default perturbation budget ϵ is set to 8/255 for CIFAR-10, CIFAR-100, SVHN, and ImageNet-30. ResNet-18, ResNet-50, Wide-ResNet, and VGG-16 are used as target models and ResNet-18 is also utilized as the pre-trained classification model P. Learning rates for target models is 10^{-1} on CIFAR-10 and CIFAR-100, and 10^{-2} on SVHN and ImageNet-30. All these networks are pre-trained and remain fixed. The denoiser D is optimized by Adam [16]. Their learning rates are initially set to 10^{-3} and decay to 8.1×10^{-6} when the training loss converges. The positive parameters λ_1 and λ_2 are respectively set to 10 and 5×10^{-1} on all datasets.

Table 9 In BPDA manner, we use PGD against our defense method with iteration numbers 20 and 40 on *CIFAR-10, CIFAR-100, SVHN, and ImagNet-30* (lower is better)

Target Model	Attack	Defense	Error rate
CIFAR-10			
DFD+ResNet18	PGD(It-20)	DFD	13.56
	PGD(It-40)		14.93
DFD′+ResNet18	PGD(It-20)	DFD′	13.14
	PGD(It-40)		14.36
CIFAR-100			
DFD+ResNet18	PGD(It-20)	DFD	51.37
	PGD(It-40)		53.16
DFD′+ResNet18	PGD(It-20)	DFD′	47.62
	PGD(It-40)		50.29
SVHN			
DFD+ResNet18	PGD(It-20)	DFD	7.53
	PGD(It-40)		8.84
DFD′+ResNet18	PGD(It-20)	DFD′	8.11
	PGD(It-40)		9.18
ImageNet-30			
DFD+ResNet18	PGD(It-20)	DFD	13.13
	PGD(It-40)		14.00
DFD′+ResNet18	PGD(It-20)	DFD′	15.27
	PGD(It-40)		16.33

DFD and DFD′ use the ResNet-18 network as the pretraining network

4.2 Defense results

Defending against unseen types of attacks. We use adversarial examples crafted by non-targeted L_2 norm C&W to train previous defense models. For C&W attack, the learining rate we set $\alpha = 10^{-2}$, the number of iterations $k = 50$, initial constant $c = 1$. We select FGSM, PGD, PGD(L_2), DeepFool, BIM, AA, DDN, STA, MI-FGSM, SI-NI-FGSM, and VMI-FGSM as unseen types of attacks to craft adversarial examples for evaluating defense models. For FGSM, PDG, AA, MI-FGSM, SI-NI-FGSM, and VMI-FGSM attacks, the attack strength we set $\epsilon = 8/255$. For PGD, AA, BIM, MI-FGSM, SI-NI-FGSM, and VMI-FGSM attacks, the iteration number we set to 40 and the attack step size $\alpha = 2/255$. For PGD(L_2) attack, we set the step to 20, the attack strength we set $\epsilon = 1$, and the attack step size $\alpha = 10^{-1}$. For the DeepFool attack, we set the iteration number to 100, and for the initial constant, we set 2×10^{-2}. As shown in Tabels

Table 10 Classification error rates (percentage) of adversarial examples (*e.g.*, PGD, MI-FGSM, and AA) crafted by various perturbations against the ResNet-18 target model on CIFAR-10 (lower is better, $\epsilon = 6/8/10/12$)

Defense	Clean	PGD				MI-FGSM				AA			
		$\epsilon = 6$	$\epsilon = 8$	$\epsilon = 10$	$\epsilon = 12$	$\epsilon = 6$	$\epsilon = 8$	$\epsilon = 10$	$\epsilon = 12$	$\epsilon = 6$	$\epsilon = 8$	$\epsilon = 10$	$\epsilon = 12$
HGD	**6.06**	10.35	15.91	22.10	28.78	11.99	14.92	25.88	33.90	9.98	12.86	21.42	26.36
APE-GAN	11.81	33.83	41.47	48.35	53.52	36.01	39.07	54.35	60.53	32.98	33.86	43.75	46.13
Comdefend	12.46	42.73	46.84	53.98	60.02	47.92	52.53	59.42	69.31	57.28	63.87	69.59	75.28
DFD	7.61	**4.23**	**5.59**	**8.62**	**13.89**	**4.24**	**5.52**	**13.63**	**23.61**	**4.62**	**5.01**	**9.29**	**12.97**
DFD′	7.58	4.64	6.01	10.03	15.38	4.64	6.42	15.90	27.49	4.79	5.45	10.05	14.68

1, 2, 3, and 4, our defense models can successfully defend against various unseen types of attacks and achieve more robust performance. As shown in Table 1, the robustness of HGD in defending against DeepFool and DDN attacks is better than our defense methods. HGD is more robust to the L_2 norm adversarial attacks on Cifar-10 since it is trained using adversarial examples crafted by non-targeted L_2 norm C&W attacks. However, our defense method achieves more robust performance against other adversarial attacks. As shown in Table 4, our defense method still achieves better defense performance even against L_2 norm adversarial attacks such as DeepFool and DDN. It is well known that the image resolution of ImageNet-30 is higher than that of CIFAR-10. It can be seen that HGD outperforms our defense method in defending against L_2 norm adversarial attacks on low-resolution datasets, but our defense method is more effective in defending against various adversarial attacks on higher-resolution datasets. Although adversarial perturbation is imperceptible at the pixel level, obvious discrepancies exist between the class activation maps of natural and adversarial examples. As shown in Fig. 3, our defense method can restore the class activation maps of various adversarial examples to be the same as those of natural examples. This shows that our approach is not specific to a single attack and it can effectively eliminate various adversarial perturbations. Meanwhile, our defense method does not lose the key information of the image.

Cross-model defense results. To evaluate the cross-model defense capability of our defense models, we transfer our denoiser (DFD), which uses the ResNet-18 network as the pretraining network, to other classification models, *i.e.*, ResNet-50, Wide-ResNet, and VGG-16. Results in Tables 5, 6, 7, and 8 present that our denoiser is significantly better than HGD and APE-GAN to eliminate adversarial perturbations crafted by PGD, AA, MI-FGSM, SI-NI-FGSM, and VMI-FGSM. As shown in Table 8, HGD is better than our defense method to eliminate adversarial perturbations crafted by C&W, but only by 1.5%. However, even against C&W attacks, our defense method exhibits better robustness than HGD and APE-GAN in Tables 6, and 7.

It illustrates that our denoiser can be deployed to different classification models. Once our defense method is trained on a particular dataset, it can be used for classification models trained on different datasets. This significantly reduces the computational cost. Therefore, our defense method is generalizable.

Defending against backward pass differentiable approximation [50] **(BPDA).** BPDA is proposed to alleviate the obfuscated gradient problem. BPDA aims to find a similar and differentiable function to replace the non-differentiable one during back-propagation. We explore the effectiveness of our defense model in the case that the attacker knows the defense model and uses BPDA to bypass it. We use PGD (attack perturbation budget $\epsilon = 16/255$, attack step size $\alpha = 2/255$) with iteration numbers 20 and 40 to attack defense models by BPDA.

As shown in Table 9, despite PGD attack using BPDA to bypass the defense, our defense method can still protect the target model, *e.g.*, the success rate of PGD attack

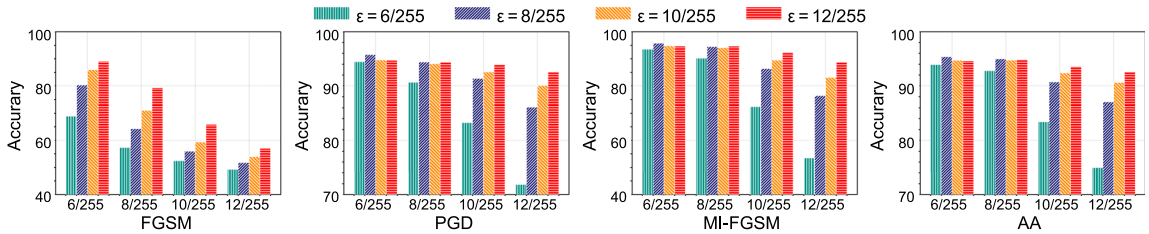

Fig. 4 Classification accuracy rates (percentage) of our defense model (training perturbation budget ϵ within the range of (6/255, 12/255)) against adversarial examples via FGSM, PGD, MI-FGSM, and AA (attack perturbation budget ϵ within the range of (6/255, 12/255))

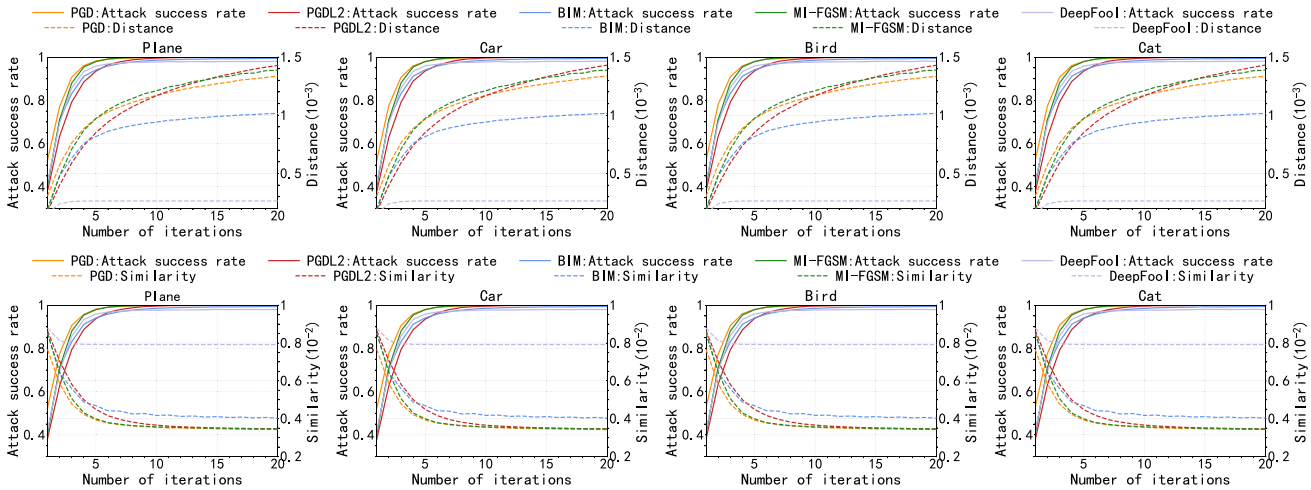

Fig. 5 The attack success rate of adversarial examples and the difference between the deep features of adversarial examples and natural examples against a ResNet-18 target model in four classes (*i.e.*, Plane, Car, Bird, and Cat) on *CIFAR-10*. The adversarial examples are crafted by PGD, PGDL_2, BIA, MI-FGSM, and DeepFool. It can be seen from the figure that the difference between the deep features increases as the attack success rate increases

with iteration number 40 is less than 17% on ImagNet-30. This indicates that our defense method is effective even when attacked in the BPDA manner.

4.3 Further evaluations

Robustness of our model to the perturbation budget. To explore the robustness of our defense model to the perturbation budget ϵ, we set the L_∞ norm perturbation budget ϵ within the range of $(6/255, 12/255)$ and craft adversarial examples via PGD, MI-FGSM, and AA. In Table 10, our defense model maintains a relatively high accuracy rate when the adversarial perturbation is constrained within $(6/255, 12/255)$. Compared to previous state-of-the-art methods, our defense model achieves the best robustness for various perturbation budgets ϵ. As the perturbation budget ϵ increases, the robustness of our defense model decreases. Therefore, we train our denoiser using perturbation budget ϵ

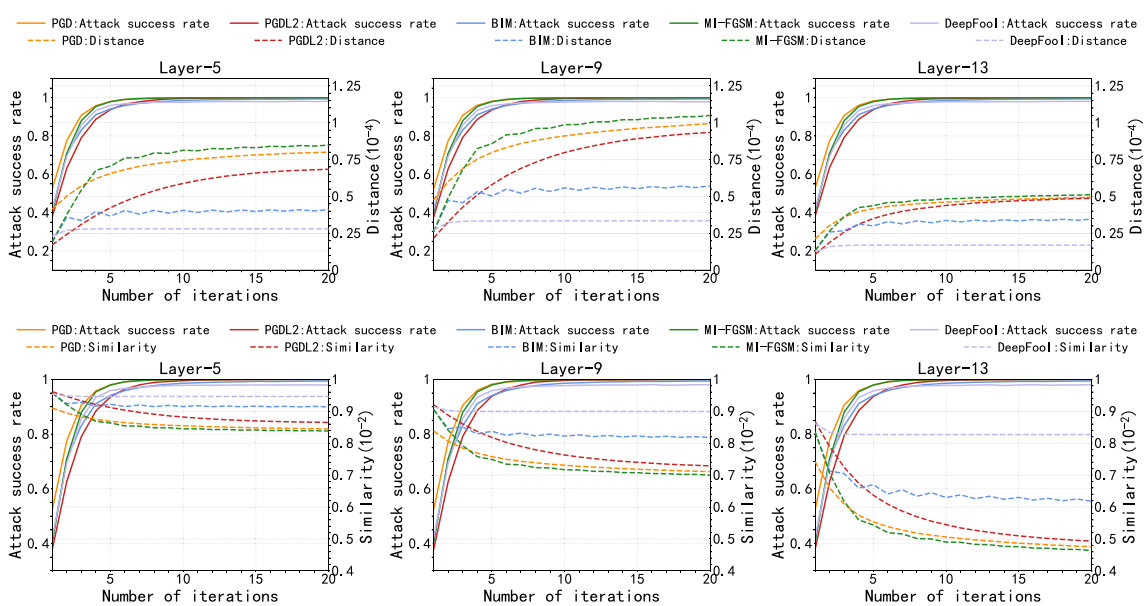

Fig. 6 The attack success rate of adversarial examples and the difference between the deep features of adversarial examples and natural examples against different layers of ResNet-18 on *CIFAR-10*.(*top*: measuring the similarity of deep features using (3), *bottom*: measuring the similarity of deep features using (4))

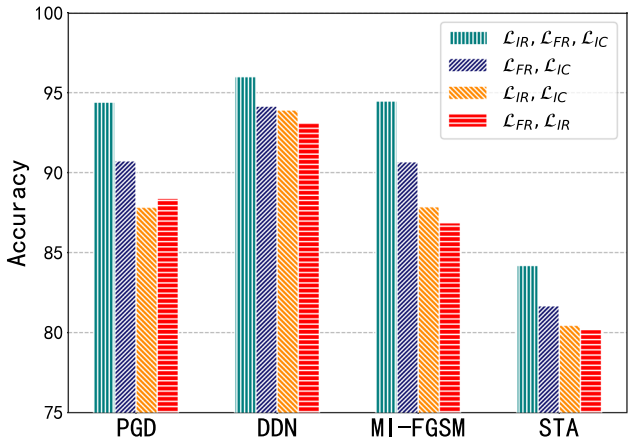

Fig. 7 Ablation study on *CIFAR-10*. The figure shows the classification accuracy rates (percentage) of ResNet-18 (higher is better). The performance of our defense method without any of the loss functions is affected

within the range of (6/255, 12/255) to defend against adversarial examples. In Fig. 4, we show our defense model trained using different perturbation budgets to defend against adversarial examples via FGSM, PGD, MI-FGSM, and AA crafted with different perturbation budgets. When the training perturbation budget is smaller than the adversarial examples' perturbation budget, our defense's robustness decreases. When the training perturbation budget is greater than or equal to the perturbation budget of the adversarial example, our defense model exhibits excellent robustness. This indicates that our defense model can be trained with a larger perturbation budget.

Specific Image Classes. In Section 3.1, we explore the commonality of various adversarial examples in DNNs in the whole dataset. We randomly select different classes from CIFAR-10 and explore whether the adversarial examples in other classes have the same commonality. In Fig. 5, we have selected four classes: Plane, Car, Bird, and Cat. It shows that even the adversarial examples crafted in different classes have the same characteristics in DNNs. The extent of distortion of deep features is proportional to the attack success rate. Meanwhile, The trends were almost consistent in different classes of datasets.

Different Network Architecture. We select the top layers of ResNet-18 to explore the common characteristics of adversarial examples in DNNs, in Section 3.1. Here, we intend to select different model architectures to explore the common characteristics. As shown in Fig. 6, even in different layers of the model, the difference between the features of adversarial examples and natural examples becomes more significant as the success rate of the attack increases. This phenomenon is particularly evident at the top layer of the network. In other words, the top layer of the network has an error amplification effect.

Ablation study. We respectively remove the image reconstruction loss L_{IR}, the feature reconstruction loss L_{FR} and the image classification loss L_{IC} to investigate their impacts on our model. Figure 7 shows the impact of the deletion loss function on classification accuracy more intuitively. Removing any of the loss functions reduces the classification accuracy.

5 Conclusion

In this paper, we designed a denoising defense to remove multiple adversarial perturbations and provide robust protections. To achieve this goal, we explore the commonality of adversarial examples and exploit this commonality to achieve defense against unknown adversarial attacks. Through experiments, we observe that adversarial examples are easily recognized and have a commonality in deep feature space. Adversarial perturbations not only disrupt the deep features of natural examples but also make their deep features progressively more dissimilar to their original selves. We exploit the commonality of adversarial examples to craft CAEs and use them to train the denoiser. Our experiments show that our defense method can remove multi-types of adversarial perturbations, and maintain the high classification accuracy of the classifier. Once the denoiser has been trained with our method, it can be deployed to different classification models and will still show good robustness. In the future, we will extend this work to applications such as object detection and tracking.

Acknowledgements This work was supported by the Major Research Plan of the National Natural Science Foundation of China (92167203), Key Program of Zhejiang Provincial Natural Science Foundation of China (LZ22F020007), the Major Key Project of Peng Cheng Laboratory (2022A03), and Science and Technology Innovation Foundation for Graduate Students of Zhejiang University of Science and Technology (F464108M05).

Author Contributions **Jianchang Huang**: Conceptualization, Methodology, Formal analysis, Investigation, Writing - original draft, Writing - review & editing, Software. **Yaguan Qian**: Conceptualization, Methodology, Formal analysis, Investigation, Writing - original draft, Writing - review & editing, Funding acquisition. **Yinyao Dai**: Data curation, Investigation, Writing - Review & Editing. **Fang Lu**: Writing - Review & Editing, Investigation. **Bin Wang**: Writing - Review & Editing, Funding acquisition. **Zhaoquan Gu**: Writing - Review & Editing. **Boyang Zhou**: Writing - Review & Editing.

Data Availability and Access The data used to support the findings of this study will be available from the corresponding author upon request after acceptance.

Declaration

Conflicts of interest The authors declare that there are no conflicts of interest regarding the publication of this paper.

Ethics Approval The data used in this paper do not involve ethical issues.

References

1. Abbas A, Abdelsamea MM, Gaber MM (2021) Classification of COVID-19 in chest x-ray images using detrac deep convolutional neural network. Appl Intell 51(2):854–864
2. Szegedy C, Zaremba W, Sutskever I, et al (2014) Intriguing properties of neural networks. In: Proc Int Conf Learn Represent (ICLR)
3. Tamou AB, Benzinou A, Nasreddine K (2021) Multi-stream fish detection in unconstrained underwater videos by the fusion of two convolutional neural network detectors. Appl Intell 51(8):5809–5821
4. Liu M, Zhang Y, Xu J et al (2021) Deep bi-directional interaction network for sentence matching. Appl Intell 51(7):4305–4329
5. Goodfellow IJ, Shlens J, Szegedy C (2015) Explaining and harnessing adversarial examples. In: Proc Int Conf Learn Represent (ICLR)
6. Yurtsever E, Lambert J, Carballo A et al (2020) A survey of autonomous driving: common practices and emerging technologies. IEEE Access 8:58443–58469
7. Pham Q, Sevestre P, Pahwa RS, et al (2020) A*3d dataset: Towards autonomous driving in challenging environments. In: 2020 IEEE international conference on robotics and automation, ICRA 2020, Paris, France, May 31 - August 31, 2020. IEEE, pp 2267–2273
8. Hoai NV, Nguyen HM, Pham C (2022) Masked face recognition with convolutional neural networks and local binary patterns. Appl Intell 52(5):5497–5512
9. Xu K, Zhang G, Liu S, et al (2020) Adversarial t-shirt! evading person detectors in a physical world. In: Proceeding European Conference Computer Vision (ECCV), Lecture Notes in Computer Science, vol 12350. Springer, pp 665–681
10. Ho J, Lee B, Kang D (2022) Attack-less adversarial training for a robust adversarial defense. Appl Intell 52(4):4364–4381
11. Tramèr F, Kurakin A, Papernot N, et al (2018) Ensemble adversarial training: attacks and defenses. In: Proc Int Conf Learn Represent (ICLR)
12. Jia X, Zhang Y, Wu B et al (2022) Boosting fast adversarial training with learnable adversarial initialization. IEEE Trans Image Process 31:4417–4430
13. Madry A, Makelov A, Schmidt L, et al (2018) Towards deep learning models resistant to adversarial attacks. In: Proc Int Conf Learn Represent (ICLR)
14. Gu S, Rigazio L (2015) Towards deep neural network architectures robust to adversarial examples. In: Proc Int Conf Learn Represent (ICLR)
15. Wang S, Gong Y (2022) Adversarial example detection based on saliency map features. Appl Intell 52(6):6262–6275
16. Sarvar A, Amirmazlaghani M (2023) Defense against adversarial examples based on wavelet domain analysis. Appl Intell 53(1):423–439
17. Guo C, Rana M, Cissé M, et al (2018) Countering adversarial images using input transformations. In: Proc Int Conf Learn Represent (ICLR)
18. Liao F, Liang M, Dong Y, et al (2018) Defense against adversarial attacks using high-level representation guided denoiser. In: Proc IEEE Conf Comput Vis Pattern Recognit (CVPR). Computer Vision Foundation / IEEE Computer Society, pp 1778–1787
19. Xu W, Evans D, Qi Y (2018) Feature squeezing: Detecting adversarial examples in deep neural networks. In: 25th annual network and distributed system security symposium, NDSS 2018, San Diego, California, USA, February 18-21, 2018. The Internet Society
20. He W, Wei J, Chen X et al (2017) (2017) Adversarial example defense: ensembles of weak defenses are not strong. In: Enck W, Mulliner C (eds) 11th USENIX Workshop on Offensive Technologies, WOOT 2017. BC, Canada, August, Vancouver, pp 14–15
21. Samangouei P, Kabkab M, Chellappa R (2018) Defense-gan: Protecting classifiers against adversarial attacks using generative models. In: Proc Int Conf Learn Represent (ICLR)
22. Jin G, Shen S, Zhang D, et al (2019) APE-GAN: adversarial perturbation elimination with GAN. In: IEEE international conference on acoustics, speech and signal processing, ICASSP 2019, Brighton, United Kingdom, May 12-17, 2019. IEEE, pp 3842–3846
23. Dong Y, Liao F, Pang T, et al (2018) Boosting adversarial attacks with momentum. In: Proc IEEE Conf Comput Vis Pattern Recognit (CVPR). Computer Vision Foundation / IEEE Computer Society, pp 9185–9193
24. Meng D, Chen H (2017) Magnet: A two-pronged defense against adversarial examples. In: Thuraisingham B, Evans D, Malkin T, et al (eds) Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017. ACM, pp 135–147
25. Carlini N, Wagner DA (2017) Magnet and "efficient defenses against adversarial attacks" are not robust to adversarial examples. arXiv:1711.08478
26. Dong Y, Liao F, Pang T, et al (2018) Boosting adversarial attacks with momentum. In: Proc IEEE Conf Comput Vis Pattern Recognit (CVPR). Computer Vision Foundation / IEEE Computer Society, pp 9185–9193
27. Moosavi-Dezfooli S, Fawzi A, Frossard P (2016) Deepfool: a simple and accurate method to fool deep neural networks. In: Proc IEEE Conf Comput Vis Pattern Recognit (CVPR). IEEE Computer Society, pp 2574–2582
28. Rony J, Hafemann LG, Oliveira LS, et al (2019) Decoupling direction and norm for efficient gradient-based L2 adversarial attacks and defenses. In: Proc IEEE Conf Comput Vis Pattern Recognit (CVPR). Computer Vision Foundation / IEEE, pp 4322–4330
29. Croce F, Hein M (2020) Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks. In: Proc Int Conf Mach Learn (ICML), Proceedings of Machine Learning Research, vol 119. PMLR, pp 2206–2216
30. Carlini N, Wagner DA (2017) Towards evaluating the robustness of neural networks. In: 2017 IEEE symposium on security and privacy, SP 2017, San Jose, CA, USA, May 22-26, 2017. IEEE Computer Society, pp 39–57
31. Dong Y, Pang T, Su H, et al (2019) Evading defenses to transferable adversarial examples by translation-invariant attacks. In: Proc IEEE Conf Comput Vis Pattern Recognit (CVPR). Computer Vision Foundation / IEEE, pp 4312–4321
32. Xie C, Zhang Z, Zhou Y, et al (2019) Improving transferability of adversarial examples with input diversity. In: Proc IEEE Conf Comput Vis Pattern Recognit (CVPR). Computer Vision Foundation / IEEE, pp 2730–2739
33. Wu K, Wang AH, Yu Y (2020) Stronger and faster wasserstein adversarial attacks. In: Proc Int Conf Mach Learn (ICML), Proceedings of Machine Learning Research, vol 119. PMLR, pp 10377–10387
34. Xiao C, Zhu J, Li B, et al (2018) Spatially transformed adversarial examples. In: Proc Int Conf Learn Represent (ICLR)
35. Tramèr F, Kurakin A, Papernot N, et al (2018) Ensemble adversarial training: Attacks and defenses. In: Proc Int Conf Learn Represent (ICLR)
36. Yan Z, Guo Y, Zhang C (2018) Deep defense: Training dnns with improved adversarial robustness. In: Bengio S, Wallach HM, Larochelle H, et al (eds) Proc Adv Neural Inf Process Syst (NeurIPS), pp 417–426
37. Xie C, Wang J, Zhang Z, et al (2018) Mitigating adversarial effects through randomization. In: Proc Int Conf Learn Represent (ICLR)

38. Jia X, Wei X, Cao X, et al (2019) Comdefend: An efficient image compression model to defend adversarial examples. In: Proc IEEE Conf Comput Vis Pattern Recognit (CVPR). Computer Vision Foundation / IEEE, pp 6084–6092
39. Xie C, Wu Y, van der Maaten L, et al (2019) Feature denoising for improving adversarial robustness. In: Proc IEEE Conf Comput Vis Pattern Recognit (CVPR). Computer Vision Foundation / IEEE, pp 501–509
40. Zhou D, Wang N, Peng C, et al (2021) Removing adversarial noise in class activation feature space. In: Proc IEEE/CVF Int Conf Comput Vis (ICCV). IEEE, pp 7858–7867
41. Zhang H, Yu Y, Jiao J, et al (2019) Theoretically principled trade-off between robustness and accuracy. In: Proc Int Conf Mach Learn (ICML), Proceedings of Machine Learning Research, vol 97. PMLR, pp 7472–7482
42. Netzer Y, Wang T, Coates A, et al (2011) Reading digits in natural images with unsupervised feature learning. nips workshop on deep learning & unsupervised feature learning
43. Deng J, Dong W, Socher R, et al (2009) Imagenet: a large-scale hierarchical image database. In: Proc IEEE Conf Comput Vis Pattern Recognit (CVPR). IEEE Computer Society, pp 248–255
44. Lin J, Song C, He K, et al (2020) Nesterov accelerated gradient and scale invariance for adversarial attacks. In: Proc Int Conf Learn Represent (ICLR)
45. Wang X, He K (2021) Enhancing the transferability of adversarial attacks through variance tuning. In: Proc IEEE Conf Comput Vis Pattern Recognit (CVPR), pp 1924–1933
46. Wu T, Tong L, Vorobeychik Y (2020) Defending against physically realizable attacks on image classification. In: Proc Int Conf Learn Represent (ICLR)
47. Simonyan K, Zisserman A (2015) Very deep convolutional networks for large-scale image recognition. In: Proc Int Conf Learn Represent (ICLR)
48. Kim H (2020) Torchattacks : a pytorch repository for adversarial attacks. arXiv:2010.01950
49. Ding GW, Wang L, Jin X (2019) advertorch v0.1: an adversarial robustness toolbox based on pytorch. arXiv:1902.07623
50. Athalye A, Carlini N, Wagner DA (2018) Obfuscated gradients give a false sense of security: circumventing defenses to adversarial examples. In: Dy JG, Krause A (eds) Proc Int Conf Mach Learn (ICML), pp 274–283

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Jianchang Huang received the B.S. degree from Dongguan University of Technology in 2021. He is now a postgraduate student of the School of Big-data Science, Zhejiang University of Science and Technology. His research interests include deep learning, machine learning security and machine vision.

Yinyao Dai received the B.S. degree from Yangzhou University in 2021. She is now a postgraduate student of the School of Big-data Science, Zhejiang University of Science and Technology. Her research interests include deep learning, machine learning security and machine vision.

Fang Lu received the Ph.D. degree in applied mathematics from Zhejiang University of China in 2016. Since 2017, she has been a teacher with the School of Big-Data Science, Zhejiang University of Science and Technology. Her main research interests include medical image processing, machine learning and optimization.

Bin Wang is a professor of Zhejiang Key Laboratory of Multi-dimensional Perception Technology, Application and Cybersecurity and Zhejiang University. He received the Ph.D. degree from the China National Digital Switching System Engineering & Technological R&D Center in 2010. His research interests mainly include Internet of Things security, cryptography, artificial intelligence security, new network security architecture and so on.

Zhaoquan Gu received his bachelor degree in Computer Science from Tsinghua University (2011) and PhD degree in Computer Science from Tsinghua University (2015). He is currently a professor in the School of Computer Science and Technology, Harbin Institute of Technology (Shenzhen), China. He is also a professor in the Department of New Networks, Peng Cheng Laboratory, Shenzhen, China. His research interests include cyberspace security, cyber range, big data analysis, and artificial intelligence security.

Boyang Zhou is currently a research expert at Intelligent Network Research Institute of Zhejiang Lab and an adjunct associate professor at Hangzhou Institute for Advanced Study, UCAS. He received his doctoral degree in Computer Science from Zhejiang University in 2014. He has published more than 30 papers and 30 patents. His research interests include Future Internet, Network Security and Smart Grid Communications.

Yaguan Qian received Ph.D. in Computer Science from Zhejiang University in 2014. He is currently a professor at the School of Science, Zhejiang University of Science and Technology. He has authored or co-authored more than 30 papers in peer-reviewed international journals and conferences, like AAAI, ECCV, IJCNN, TOPS, and COSE. His main research interests include AI security, machine learning, pattern recognition, and machine vision.

Authors and Affiliations

Jianchang Huang[1] · Yinyao Dai[1] · Fang Lu[1] · Bin Wang[2] · Zhaoquan Gu[3] · Boyang Zhou[4] · Yaguan Qian[1]

Jianchang Huang
212109701007@zust.edu.cn

Yinyao Dai
222109252002@zust.edu.cn

Fang Lu
lufang218@126.com

Bin Wang
wangbin2@hikvision.com

Zhaoquan Gu
guzhaoquan@hit.edu.cn

Boyang Zhou
zhouby@zhejianglab.com

1 School of Science, Zhejiang University of Science and Technology, Hangzhou 3210023, Zhejiang, China

2 Application and Cybersecurity, Zhejiang Key Laboratory of Multi-dimensional Perception Technology, Hangzhou 310051, Zhejiang, China

3 School of Computer Science and Technology, Harbin Institute of Technology (Shenzhen), Shenzhen 518055, Guangdong, China

4 Intelligent Network Institute, Zhejiang Lab, Hangzhou 311121, Zhejiang, China